

GovJobs Application Pipeline and Technology Architecture

Overview

GovJobs is a hybrid system combining Python backend services, a Next.js frontend, and an ingestion pipeline for job notice extraction. It stores data in PostgreSQL, uses Redis for queues and caching, and can optionally perform LLM-driven PDF extraction through Ollama.

Backend

- FastAPI serves the REST API from `app/main.py`.
- Uvicorn runs the ASGI web server.
- SQLAlchemy manages ORM models and PostgreSQL access.
- Alembic handles migrations.
- Celery processes scheduled and async tasks.
- Redis is the Celery broker and queue backend.
- SQLAlchemy provides an admin panel.
- Key backend modules:
 - `app/api/` routes for authentication, jobs, admin actions, image converter, compress and misc APIs.
 - `app/crawler/` contains source fetchers, parsers, and the raw-item/job upsert pipeline.
 - `app/tasks/` defines Celery tasks for crawling, job enrichment, status refresh, and archival.
 - `app/services/` includes job lifecycle, source registry, enrichment, PDF and notice parsing logic.
 - `app/models/` defines database entities such as `Job`, `RawItem`, `Source`, `AuditLog`, `DedupeKey`.

Frontend and Ingestion

- Next.js (React) provides the frontend UI.
- TypeScript is used for frontend code.
- Prisma is the database client and migration tool for the web project.
- BullMQ implements the ingestion queue in the web layer.
- Playwright and Cheerio are used for headless page fetch and HTML parsing.
- PDF parsing uses `pdf-parse` and the backend Python stack for PDF extraction support.
- Key ingestion files:
 - `web/src/ingest/queue.ts`: configures BullMQ and the Redis connection.
 - `web/src/ingest/worker.ts`: runs jobs for source ingestion, PDF download, and normalization.
 - `web/src/ingest/scheduler.ts`: schedules ingest cycles and normalization jobs.
 - `web/src/ingest/ingestOneSource.ts`: crawls source list pages and creates raw notifications.
 - `web/src/ingest/normalize.ts`: fetches detail pages, extracts PDF text, runs regex and LLM extraction, and upserts jobs.
 - `web/src/ingest/llmDetailsExtractor.ts`: integrates with Ollama to extract structured fields from PDF/text.
 - `web/src/ingest/detailsExtractor.ts`: deterministic regex-based extraction fallback.
 - `web/src/ingest/pdfText.ts`: pulls text from PDFs.

Ingestion Pipeline Flow

1. Source discovery:

- `DEFAULT_SOURCES` are seeded from `web/src/ingest/sources.ts`.

- ``ingest:seed`` populates the source list.

2. Schedule ingestion:

- ``scheduler.ts`` enqueues ``ingest-source``, ``drain-downloads``, and ``normalize-latest`` jobs.
- ``ingest:dev`` starts the ingestion worker and scheduler.

3. List page crawl:

- ``ingestOneSource.ts`` fetches and parses announcements from government listings.
- New items become ``rawNotification`` entries in the database.

4. PDF download and text extraction:

- ``queue.ts`` schedules ``download-pdf`` jobs for raw items with PDF URLs.
- PDF text is extracted and stored.

5. Normalization:

- ``normalize.ts`` selects raw entries and processes them.
- It may fetch detail pages (especially UPSC and LLM-enabled sources).
- It applies regex parsing to PDF and page text.
- If enabled, it calls Ollama via ``llmDetailsExtractor.ts``.

6. LLM-first extraction:

- The system sends normalized notice context to Ollama at ``OLLAMA_BASE_URL``.
- The Qwen model returns a strict JSON object with fields like apply dates, vacancy, exam dates, and URLs.
- The result is validated, merged with deterministic extraction, and saved.

7. Job creation:

- Processed raw notifications are merged into ``job`` and ``jobDetails`` entries.
- If a notice is non-public or not a job, it may be marked processed without job creation.

LLM Extraction Details

- Enabled with ``LLM_EXTRACTOR_ENABLED=true`` in ``web/.env``.
- Uses Ollama model ``qwen2.5:7b`` by default.
- Extracts structured fields from PDF text, detail page text, and parsed URLs.
- Falls back to deterministic regex extraction if the LLM returns no usable output.
- Stores extraction metadata, confidence, prompt version, and raw model response in the database.

Technology Stack

- Python backend:
- FastAPI, Uvicorn, SQLAlchemy, Alembic, Pydantic, Celery, Redis, SQLAlchemyAdmin.
- Data extraction: ``beautifulsoup4``, ``lxml``, ``httpx``, ``pdfplumber``, ``PyPDF2``, ``pypdfium2``, ``PaddleOCR``.
- Frontend / ingestion:

- Next.js, React, TypeScript, Tailwind CSS, Prisma, BullMQ, Axios, Cheerio, Playwright, Zod.
- Database:
- PostgreSQL.
- Queue / cache:
- Redis.
- LLM integration:
- Ollama with Qwen model.

Run Commands

Backend:

- ``source myvenv/bin/activate``
- ``pip install -r requirements.txt``
- ``alembic upgrade head``
- ``uvicorn app.main:app --reload --host 0.0.0.0 --port 8000``
- ``celery -A app.tasks.celery_app worker -l info``
- ``celery -A app.tasks.celery_app beat -l info``

Web + ingestion:

- ``cd web``
- ``npm install``
- ``npm run prisma:generate``
- ``npm run prisma:migrate``
- ``npm run ingest:seed``
- ``npm run dev``
- ``npm run ingest:dev``

LLM PDF extraction:

- ``ollama pull qwen2.5:7b``
- set env vars in ``web/.env``.